

OSSP_SIGNALS

2520030395

Monish Putti

Signal.c

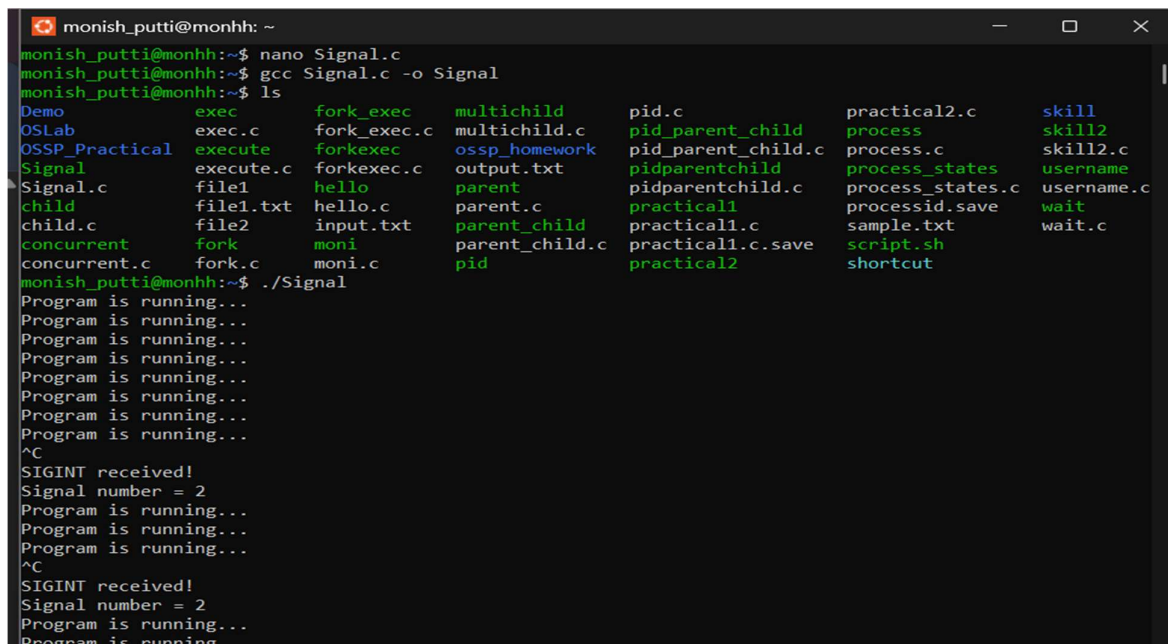
```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    printf("\nSIGINT received!\n");
    printf("Signal number = %d\n", sig);
}

int main()
{
    signal(SIGINT, handler);

    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }

    return 0;
}
```



```
monish_putti@monhh: ~
monish_putti@monhh:~$ nano Signal.c
monish_putti@monhh:~$ gcc Signal.c -o Signal
monish_putti@monhh:~$ ls
Demo      exec      fork_exec  multichild  pid.c      pid_parent_child  practical2.c  skill
OSLab     exec.c    fork_exec.c  multichild.c  pid_parent_child.c  process.c          process_states  skill2.c
OSSP_Practical  execute   forkexec     ossp_homework  pidparentchild  process_id.save    process_id.save  username
Signal.c   file1.txt hello.c      parent.c      practical1      practical1.c.save  sample.txt      wait
child.c    file2.txt input.txt    parent_child  practical1.c.save  script.sh          shortcut
concurrent  fork      moni.c      pid
concurrent.c  fork.c    moni.c      pid

monish_putti@monhh:~$ ./Signal
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
SIGINT received!
Signal number = 2
Program is running...
Program is running...
Program is running...
Program is running...
^C
SIGINT received!
Signal number = 2
Program is running...
Program is running...
```

1. Sigterm

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    if (sig == SIGTERM)
        printf("\nSIGTERM received!\n");
}

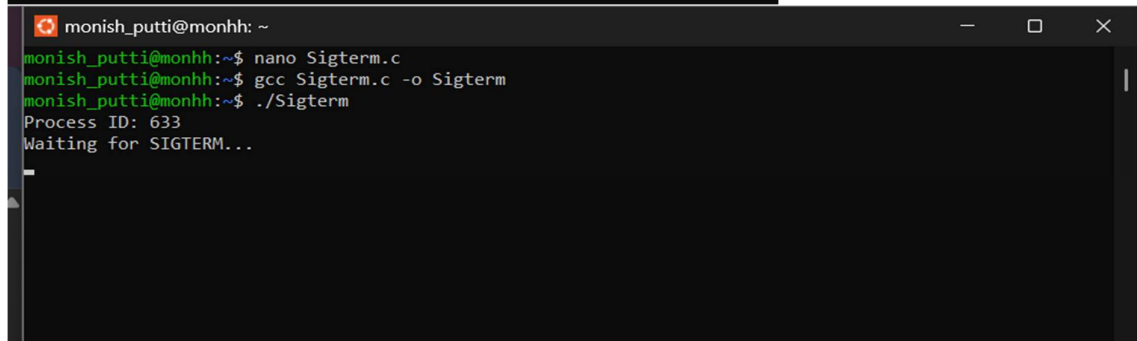
int main()
{
    signal(SIGTERM, handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for SIGTERM...\n");

    while (1)
    {
        sleep(2);
    }

    return 0;
}

```



```

monish_putti@monhh: ~
monish_putti@monhh:~$ nano Sigterm.c
monish_putti@monhh:~$ gcc Sigterm.c -o Sigterm
monish_putti@monhh:~$ ./Sigterm
Process ID: 633
Waiting for SIGTERM...

```

2. Sigchild

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

void child_handler(int sig)
{
    printf("SIGCHLD received: Child has terminated\n");
    wait(NULL);
}

int main()
{
    signal(SIGCHLD, child_handler);

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child is running...\n");
        sleep(2);
        printf("Child is exiting...\n");
    }
    else
    {
        printf("Parent is waiting...\n");
        sleep(5);
        printf("Parent finished.\n");
    }

    return 0;
}

```

```
monish_putti@monhh: ~  
monish_putti@monhh:~$ nano Sigterm.c  
monish_putti@monhh:~$ gcc Sigterm.c -o Sigterm  
monish_putti@monhh:~$ ./Sigterm  
Process ID: 633  
Waiting for SIGTERM...  
^C  
monish_putti@monhh:~$ nano Sigchild.c  
monish_putti@monhh:~$ gcc Sigchild.c -o Sigchild  
monish_putti@monhh:~$ ./Sigchild  
Parent is waiting...  
Child is running...  
Child is exiting...  
SIGCHLD received: Child has terminated  
Parent finished.  
monish_putti@monhh:~$
```

3. Sigpipe

```
#include <stdio.h>  
#include <unistd.h>  
#include <signal.h>  
#include <stdlib.h>  
  
void handle_sigpipe(int sig)  
{  
    printf("\nSIGPIPE received!\n");  
    printf("The reading end of the pipe is closed.\n");  
    exit(1);  
}  
  
int main()  
{  
    int fd[2];  
  
    signal(SIGPIPE, handle_sigpipe);  
  
    pipe(fd);  
  
    printf("Pipe created.\n");  
  
    // Close the reading end  
    close(fd[0]);  
  
    printf("Reading end closed.\n");  
  
    printf("Trying to write to the pipe...\n");  
  
    write(fd[1], "Hello", 5);  
  
    printf("Write completed.\n");  
  
    return 0;  
}  
  
SIGCHLD received: Child has terminated  
Parent finished.  
monish_putti@monhh:~$ nano Sigpipe.c  
monish_putti@monhh:~$ gcc Sigpipe.c -o Sigpipe  
monish_putti@monhh:~$ ./Sigpipe  
Pipe created.  
Reading end closed.  
Trying to write to the pipe...  
  
SIGPIPE received!  
The reading end of the pipe is closed.  
monish_putti@monhh:~$
```

4. Sigsegv

```

#include <stdio.h>
#include <signal.h>
#include <stdlib.h>

void handle_sigsegv(int sig)
{
    printf("\nSIGSEGV received!\n");
    printf("Invalid memory access detected.\n");
    exit(1);
}

int main()
{
    signal(SIGSEGV, handle_sigsegv);

    printf("Program started.\n");

    int *p = NULL;

    printf("Trying to access invalid memory...\n");

    *p = 10;

    printf("Program completed.\n");

    return 0;
}

```

Trying to write to the pipe...

```

SIGPIPE received!
The reading end of the pipe is closed.
monish_putti@monhh:~$ nano Sigsegv.c
monish_putti@monhh:~$ gcc Sigsegv.c -o Sigsegv
monish_putti@monhh:~$ ./Sigsegv
Program started.
Trying to access invalid memory...

SIGSEGV received!
Invalid memory access detected.
monish_putti@monhh:~$ █

```